

REPORT

Question

Edge Coloring Problem (NP-Complete Problem)

Aim

To develop a backtracking-based solution for the Edge Coloring Problem by assigning colors to edges such that no two adjacent edges have the same color.

Problem Statement

Given a graph, assign colors to its edges such that:

No two edges sharing the same vertex can have the same color.

A limited number of colors is provided. The objective is to determine whether a valid edge coloring exists using the available colors.

Introduction

Edge coloring is a graph-coloring problem in which colors are assigned to edges rather than vertices.

Two edges are called **adjacent** if they share a common vertex.

For example:

Edge 1: (1,2)

Edge 2: (1,3)

These edges are adjacent because both contain vertex 1.

Therefore, they must have different colors.

Constraints

The main constraints are:

1. Adjacent Edges

Two edges sharing a vertex cannot have the same color.

For edges:

(u,v)

(x,y)

they are adjacent if:

$$u = x$$

or

$$u = y$$

or

$$v = x$$

or

$$v = y$$

2. Color Limit

Only a fixed number of colors can be used.

If the number of available colors is:

$$k$$

then every edge must receive a color from:

$$1, 2, \dots, k$$

3. Every Edge Must Be Colored

A valid solution must assign exactly one valid color to every edge.

Example Graph

Consider the following edges:

(1,2)
(1,3)
(2,3)
(2,4)
(3,4)

Using three colors, a valid assignment is:

(1,2) → Color 1
(1,3) → Color 2
(2,3) → Color 3
(2,4) → Color 2
(3,4) → Color 1

No two edges sharing a vertex have the same color.

Backtracking Approach

Backtracking constructs the solution incrementally.

For every edge:

1. Try the first available color.
 2. Check whether the color creates a conflict.
 3. If there is no conflict, assign the color.
 4. Move to the next edge.
 5. If no color works, undo the previous assignment.
 6. Try another color.
 7. Continue until all edges are colored or all possibilities fail.
-

Constraint Checking

Before assigning a color, the algorithm checks all previously colored edges.

Suppose:

Edge A = (1,2)
Color = 1

and we want to color:

Edge B = (1,3)

Since both edges share vertex 1, they are adjacent.

Therefore:

Edge B cannot use Color 1.

The algorithm rejects that assignment and tries another color.

Backtracking Example

Suppose there are three colors:

Color 1
Color 2
Color 3

The algorithm may initially assign:

Edge 1 $\rightarrow$ Color 1

Then:

Edge 2 $\rightarrow$ Color 1

If Edge 1 and Edge 2 are adjacent, this creates a conflict.

The algorithm therefore backtracks:

Edge 2 $\rightarrow$ Color 2

If this is valid, it continues.

This process prevents invalid partial solutions from being extended unnecessarily.

Algorithm

EdgeColoring(edgeIndex):

 If edgeIndex == number of edges:
 return TRUE

 For every available color:
 If color is safe for edge:
 Assign color

 If EdgeColoring(edgeIndex + 1):
 return TRUE

 Remove color assignment

 return FALSE

Correctness

The algorithm is correct because every color assignment is checked against all previously colored adjacent edges.

Case 1 – Safe Color

If a color does not conflict with any adjacent edge, it is assigned and the algorithm continues.

Case 2 – Conflict

If the color conflicts with an adjacent edge, it is rejected.

Case 3 – No Color Available

If none of the available colors can be assigned to the current edge, the algorithm backtracks to the previous edge and tries another color.

Therefore, if a valid coloring exists within the specified color limit, the algorithm can find it.

If the algorithm explores all possible assignments and none succeeds, no valid coloring exists with that number of colors.

Time Complexity

Suppose:

- E = number of edges
- K = number of available colors

Each edge can potentially be assigned one of K colors.

Therefore, the number of possible assignments can be approximately:

$$K^E$$

For each assignment, the algorithm performs constraint checking against previously colored edges.

This can add an $O(E)$ factor.

Thus, a simple upper-bound analysis is:

$$\boxed{O(E \times K^E)}$$

The exponential nature comes from the backtracking search.

Space Complexity

The algorithm stores:

- Edge information: $O(E)$
- Color assignments: $O(E)$
- Recursion stack: $O(E)$

Therefore:

$$\boxed{O(E)}$$

additional algorithmic space, excluding the input graph storage.

NP-Complete Complexity

The decision version of the general edge-coloring problem asks whether the edges of a graph can be colored using at most k colors.

For general graphs, this problem is computationally difficult.

The edge-coloring problem is closely related to graph coloring through the **line graph** transformation: edges of the original graph become vertices of the line graph, and adjacent original edges become adjacent line-graph vertices.

Thus, general constrained coloring can require exponential search, and backtracking has exponential worst-case behavior.

Important terminology: the exact complexity classification depends on the formulation. The general **k -edge-colorability decision problem** is NP-complete for appropriate unrestricted values of k ; special cases such as determining the minimum number of edge colors have additional graph-theoretic structure.

Effect of Constraints

The number of available colors strongly affects the search.

More Colors

More colors generally make it easier to find a valid assignment.

For example:

3 colors $\rightarrow$ potentially many valid choices
5 colors $\rightarrow$ even more choices

Fewer Colors

Fewer colors increase the chance of conflicts.

For example, a highly connected graph may require more colors.

Therefore, constraints directly affect the size of the feasible solution space.

Advantages

- Simple to understand.
 - Finds a valid solution when one exists.
 - Explicitly checks constraints.
 - Can solve small and medium-sized instances.
 - Demonstrates the general backtracking technique.
-

Limitations

- Worst-case execution time is exponential.
 - Performance decreases rapidly as the number of edges increases.
 - Highly connected graphs may require extensive backtracking.
 - Not ideal for extremely large graphs.
-

Scalability

For small graphs, backtracking is practical.

However, as the number of edges increases:

$$K^E$$

can grow extremely quickly.

For example:

$$3^5 = 243$$

but:

$$3^{20} = 3,486,784,401$$

possible color assignments exist before considering pruning.

Therefore, constraint checking and early backtracking are important for reducing unnecessary exploration.

Result

The Edge Coloring Problem was successfully implemented using backtracking.

The program assigns colors to edges while checking adjacency constraints before every assignment.

For the sample graph, a valid coloring can be found using three colors.

The algorithm has exponential worst-case behavior, approximately:

$$O(EK^E)$$

with the implemented constraint-checking strategy.

Conclusion

The Edge Coloring Problem demonstrates how backtracking can solve constraint-satisfaction problems.

The algorithm incrementally assigns colors, checks conflicts, and backtracks whenever a partial assignment cannot lead to a valid solution.

Although the approach is effective for smaller graphs, its exponential worst-case complexity limits scalability for very large graphs.

The problem also demonstrates the importance of constraint checking because invalid assignments can be eliminated early, reducing unnecessary exploration.